Module 1 - Playwright Setup & Basics

1. What is Playwright?

Playwright is an **end-to-end automation testing tool** built by **Microsoft**. It allows testers and developers to **automate browsers** like:

- Chromium (Google Chrome, Microsoft Edge)
- Firefox
- WebKit (Safari)

It's mainly used for:

- Functional testing
- UI testing
- Regression testing
- Cross-browser compatibility testing

What is Playwright (TypeScript Version)

Playwright + **TypeScript** = a modern, strongly-typed automation framework that gives you:

- IntelliSense (code suggestions & auto-completion)
- Compile-time error checking
- Better readability for large projects
- Strong object model for Page Objects and Fixtures

2. Why Playwright?

Let's understand **why most modern testers prefer Playwright**:

Feature	Description	Real-Life Benefit
Cross-browser support	Works on Chrome, Firefox, and Safari	One script works everywhere
Auto-waiting	Waits automatically for elements	Reduces flakiness
Parallel Execution	Runs multiple tests at once	Saves testing time
Supports multiple languages	JavaScript, TypeScript, Python, C#, Java	Easy integration
Headless mode	Runs without showing browser window	Ideal for CI/CD pipelines
Video & screenshot recording	Captures during test	Helps debug issues
Powerful locators	Can detect elements like getByRole(), getByText()	Stable selectors

3. Prerequisites

Before installing Playwright, ensure your system is ready:

System Requirements:

- Node.js (Version 16 or above)
- npm (Node Package Manager)
- A text editor like **VS Code**
- Internet connection

4. Step-by-Step Installation Guide

Step 1: Create Project Folder

```
mkdir playwright-learning
cd playwright-learning
```

Step 2: Initialize Node Project

```
npm init -y
```

This creates a **package.json** file - which stores all dependencies and metadata about your project.

Step 3: Install Playwright

```
npm init playwright@latest
```

During installation, Playwright will ask questions:

Question	Example Answer
Which language?	JavaScript
Add example tests?	Yes
Install Playwright browsers?	Yes

Once finished, it will automatically create all required files.

Step 4: Check Installed Version

```
npx playwright --version
```

Output Example:

```
Version 1.48.0
```

Step 5: Run Sample Test

```
npx playwright test
```

You'll see something like:

```
Running 3 tests using 3 workers
✔ example.spec.js › has title
✔ example.spec.js › get started link
✔ example.spec.js › navigation test
```

Congratulations! Playwright is working perfectly.

5. Understanding Folder Structure

After installation, your folder will look like this:

```
playwright-ts-demo/
│
├── tests/
│   └── example.spec.ts
│
├── playwright.config.ts
├── package.json
├── package-lock.json
├── tsconfig.json
├── test-results/
├── playwright-report/
└── node_modules/
```

Folder & File Explanation

Item	Description
`tests/`	Contains all `.spec.ts` test files
`example.spec.ts`	Default sample test
`playwright.config.ts`	Global Playwright configuration
`tsconfig.json`	TypeScript compiler options
`test-results/`	Stores videos/screenshots/logs
`playwright-report/`	HTML reports
`node_modules/`	Dependencies

6. Understanding TypeScript Test File

Let's create a new test to **open the browser and verify page title**.

Create a New Test File

File: *`tests/example.spec.ts`*

```ts
import { test, expect } from '@playwright/test';

test('basic navigation test', async ({ page }) => {
  // Step 1 - Open Playwright website
  await page.goto('https://playwright.dev/');

  // Step 2 - Check page title
  const title = await page.title();

  // Step 3 - Print title
  console.log('Title:', title);

  // Step 4 - Validate title text
  await expect(title).toContain('Playwright');
});
```

AI Driven

Real-Time Explanation

Line	Description
import { test, expect }	Imports Playwright's testing functions
async ({ page })	page fixture represents a browser tab
page.goto()	Navigates to given URL
page.title()	Retrieves current page title
expect()	Assertion to validate results

Running Tests

Run All Tests

```
npx playwright test
```

Run Specific Test File

```
npx playwright test tests/example.spec.ts
```

Run in Headed Mode (see browser)

```
npx playwright test --headed
```

Generate and View Report

```
npx playwright show-report
```

It opens `playwright-report/index.html` with:

- Passed/failed tests
- Screenshots & videos
- Execution time

7. Playwright Config File (playwright.config.ts)

Example:

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  timeout: 30 * 1000,                    // 30 seconds
  retries: 1,                            // Retry once if failed
  use: {
    headless: true,                      // Run without UI
    screenshot: 'only-on-failure',       // Capture on failure
    video: 'retain-on-failure',          // Record failed tests
  },
  projects: [
    { name: 'Chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'Firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'WebKit', use: { ...devices['Desktop Safari'] } },
  ],
});
```

Explanation

- `testDir` → Location of test files
- `retries` → How many times to rerun failed tests
- `use` → Global settings (headless, video, screenshots)
- `projects` → Cross-browser testing setup

8. TypeScript Configuration (tsconfig.json)

Created automatically.

Typical content:

```
{
  "compilerOptions": {
    "target": "ESNext",
    "module": "commonjs",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  }
}
```

This ensures:

- Modern JS support
- Type checking for Playwright API
- Clean transpilation

9. Creating Your Own First TypeScript Test

Create file `tests/firstTest.spec.ts`

```
import { test, expect } from '@playwright/test';

test('verify GitHub home page title', async ({ page }) => {

  // Step 1: Navigate
  await page.goto('https://github.com');

  // Step 2: Fetch title
  const title = await page.title();
  console.log('GitHub Title:', title);

  // Step 3: Assert
  await expect(title).toContain('GitHub');
});
```

Run it:

```
npx playwright test tests/firstTest.spec.ts --project=Chromium
```

Output:

```
✔ tests/firstTest.spec.ts › verify GitHub home page title (2s)
```

10. How Playwright Test Runner Works (Real Explanation)

When you run:

```
npx playwright test
```

Playwright does the following internally:

1. Loads `playwright.config.ts`
2. Starts multiple browser workers (parallel execution)
3. Opens each `.spec.ts` file
4. Launches browser context for each test
5. Executes steps like `goto`, `click`, `fill`, etc.
6. Collects logs, screenshots, videos
7. Generates `playwright-report`

AI Driven

11. Common CLI Commands

Command	Purpose
npx playwright test	Run all tests
npx playwright test --headed	Run with visible browser
npx playwright test --debug	Open debug mode
npx playwright show-report	Open HTML report
npx playwright codegen <url>	Record UI actions and generate TypeScript code
npx playwright install	Re-install browsers
npx playwright test --trace on	Record trace for debugging

12. Real-World Example Scenario

Project: E-commerce Login Validation

```
import { test, expect } from '@playwright/test';

test('verify login page title in ShopEase', async ({ page })
=> {
  await page.goto('https://demo.opencart.com/');
  const title = await page.title();
  console.log('Page title:', title);
  await expect(title).toContain('Your Store');
});
```

Run with:

```
npx playwright test tests/shopEase.spec.ts --headed
```

In real CI/CD pipeline:

- Runs headless for speed
- Saves screenshots on failure
- Generates report for team

13. Common TypeScript Playwright Errors & Fixes

Issue	Cause	Fix
Cannot find module '@playwright/test'	Missing dependency	npm install @playwright/test
Browser not launching	Browsers not installed	npx playwright install
Timeout Error	Page slow to load	Increase timeout in config
Tests not found	Wrong folder or extension	Must be *.spec.ts inside /tests

14. Best Practices (For TypeScript Automation)

1. Use **async/await** properly in every test
2. Keep tests **independent** (no shared state)
3. Use **fixtures** for setup/teardown
4. Enable **trace viewer** for debugging
5. Store selectors in separate Page Object files
6. Use meaningful test names
7. Run tests in CI (headless mode)